HYPERPARAMETER HANDS-ON

Linear Regression

- **Model:**
  Linear Regression predicts continuous values by fitting a linear relationship between input features and output. It assumes linearity, independence, and constant variance.
- **Hyperparameters:**
- **fit_intercept** – Whether to include intercept in model
- **normalize** – Normalize input features before training
- **positive** – Forces coefficients to be positive
- **copy_X** – Copies input matrix or not
- **n_jobs** – Number of CPU cores used

Logistic Regression

- **Model:**
  Logistic Regression predicts probability of categorical outcomes using sigmoid function. It is widely used for binary classification.
- **Hyperparameters:**
- **penalty** – Type of regularization (l1, l2, elasticnet)
- **C** – Inverse of regularization strength
- **solver** – Optimization algorithm
- **max_iter** – Maximum iterations for convergence
- **class_weight** – Handles imbalanced classes

Decision Tree

- **Model:**
  Decision Tree splits data recursively using feature thresholds. It works for both classification and regression.
- **Hyperparameters:**
- **max_depth** – Maximum depth of tree
- **min_samples_split** – Minimum samples to split node
- **min_samples_leaf** – Minimum samples at leaf
- **criterion** – Split quality metric
- **max_features** – Features considered per split

Random Forest

- **Model:**
  Random Forest builds multiple decision trees and averages their predictions. It reduces overfitting and improves stability.
- **Hyperparameters:**
- **n_estimators** – Number of trees
- **max_depth** – Depth of trees
- **min_samples_split** – Minimum samples to split
- **min_samples_leaf** – Minimum samples at leaf
- **max_features** – Features per split

Gradient Boosting

- **Model:**
  Gradient Boosting builds trees sequentially where each tree corrects previous errors.
- **Hyperparameters:**
- **n_estimators** – Number of boosting stages
- **learning_rate** – Shrinks contribution of each tree
- **max_depth** – Depth of trees
- **subsample** – Fraction of samples used
- **min_samples_split** – Split requirement

XGBoost

- **Model:**
  XGBoost is an optimized gradient boosting algorithm with regularization and parallel processing.
- **Hyperparameters:**
- **n_estimators** – Number of trees
- **learning_rate** – Step size shrinkage
- **max_depth** – Tree depth
- **subsample** – Row sampling ratio
- **colsample_bytree** – Feature sampling ratio
- **gamma** – Minimum loss reduction

CatBoost

- **Model:**
  CatBoost handles categorical variables automatically and uses ordered boosting to reduce overfitting.
- **Hyperparameters:**
- **iterations** – Number of boosting rounds
- **learning_rate** – Step size
- **depth** – Tree depth
- **l2_leaf_reg** – L2 regularization
- **loss_function** – Optimization objective

LightGBM

- **Model:**
  LightGBM is a fast gradient boosting framework using leaf-wise tree growth.
- **Hyperparameters:**
- **num_leaves** – Maximum leaves per tree
- **learning_rate** – Step size
- **n_estimators** – Number of trees
- **max_depth** – Tree depth
- **min_data_in_leaf** – Minimum samples in leaf

Support Vector Machine (SVM)

- **Model:**
  SVM finds optimal hyperplane that maximizes margin between classes.
- **Hyperparameters:**
- **kernel** – Transformation function
- **C** – Regularization parameter
- **gamma** – Kernel coefficient
- **degree** – Polynomial kernel degree

K-Nearest Neighbors (KNN)

- **Model:**
  KNN predicts output using nearest neighbors in feature space.
- **Hyperparameters:**
- **n_neighbors** – Number of neighbors
- **weights** – Uniform or distance based
- **metric** – Distance function
- **p** – Minkowski power parameter

Naive Bayes

- **Model:**
  Naive Bayes is a probabilistic classification algorithm based on Bayes' theorem. It assumes independence between features and works well for text classification and high-dimensional data.
- **Hyperparameters:**
- **var_smoothing** – Portion of variance added to features for numerical stability (GaussianNB).
- **alpha** – Additive smoothing parameter to avoid zero probability (MultinomialNB).
- **fit_prior** – Whether to learn class prior probabilities.
- **class_prior** – Predefined prior probabilities for classes.
- **binarize** – Threshold for binarizing features (BernoulliNB).

AdaBoost

- **Model:**
  AdaBoost is an ensemble boosting algorithm that combines weak learners (usually decision trees). Each new model focuses more on previously misclassified samples.
- **Hyperparameters:**
- **n_estimators** – Number of weak learners used in boosting.
- **learning_rate** – Shrinks contribution of each classifier.
- **base_estimator** – Weak learner used (typically decision tree stump).
- **algorithm** – SAMME or SAMME.R boosting algorithm.
- **random_state** – Controls randomness for reproducibility.